GuardMyCode report

Client: Example Co

Date: 2026-10-16 19:58 UTC

Projects reviewed: tienda-en-linea

1. Summary for the owner

F

RISK GRADE

Customer data open to the public

5	9	4	2
Critical	High	Medium	Low

Grade before fixes. A: no serious issues. B: only medium issues. C: high-risk issues open. D: critical issues open. F: customer data open to the public.

Most urgent problems:

1. **Firebase rules allow anyone access (if true)** (critical, same day) — `firestore.rules:4`. Require a signed-in user and check ownership, for example: allow read, write: if request.auth.uid == userId;
2. **AWS access key** (critical, same day) — `.env:1`. Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
3. **Database URL with password** (critical, same day) — `.env:2`. Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
4. **Stripe live secret key** (critical, same day) — `billing/stripe.js:1`. Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
5. **Supabase service_role key (full database access)** (critical, same day) — `src/lib/supabase.ts:1`. Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.

+9 more findings in the table below

2. Inventory

Project	Files	Languages	AI tools and platforms detected	People who committed	Commits	Last change	Commits marked as written with AI	Where the code is hosted
tienda-en-linea	10	Python, JavaScript, TypeScript, SQL	Claude Code, Supabase, Firebase	1	3	2026-10-16	Claude 1	—

3. Findings

#	Severity	Area	Issue	Where	Evidence (masked)	How to fix
1	**Critical**	Outside exposure	Firebase rules allow anyone access (if true) Anyone on the internet can read or change this data without logging in.	firestore.rules:4	—	Require a signed-in user and check ownership, for example: allow read, write: if request.auth.uid == userId;
2	**Critical**	Secrets	AWS access key Found in the code.	.env:1	AK...****** (20 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
3	**Critical**	Secrets	Database URL with password Found in the code.	.env:2	****** (10 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
4	**Critical**	Secrets	Stripe live secret key Found in the code.	billing/stripe.js:1	sk_l...******...mJ (32 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
5	**Critical**	Secrets	Supabase service_role key (full database access) Found in the code.	src/lib/supabase.ts:1	eyJh...******...r3 (136 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
6	**High**	AI tool settings	Claude Code set to skip all permission prompts The AI agent can run any command, edit any file and reach the network without asking, on whatever machine opens this project.	.claude/settings.json	—	Remove defaultMode bypassPermissions; allow only the specific commands the project needs.
7	**High**	AI tool settings	Hidden characters in AI instructions file (3) Invisible characters in files that instruct AI coding tools can hide instructions from human reviewers ('rules file backdoor').	CLAUDE.md:2	U+200B, U+202C, U+202E	Inspect the file with a hex viewer, remove the hidden characters, and find out who added them.
8	**High**	Code flaws	SQL built by joining text (possible injection) User input placed directly into SQL can let attackers read or change the database.	api/app.py:3	—	Use parameterised queries or the ORM's query builder.
9	**High**	Outside exposure	Secret exposed to the browser through NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY Variables with this prefix are bundled into the website and visible to every visitor.	src/lib/supabase.ts:2	—	Keep secrets on the server only; call them from a backend route or server function instead.
10	**High**	Outside exposure	Table 'customers' has no row-level security On Supabase, tables in the public schema without row-level security can be read and edited by anyone holding the public anon key, which ships inside the website. This is the flaw behind CVE-2025-48757, which exposed data in 170+ AI-built apps.	supabase/migrations/001_init.sql:1	—	Run: alter table customers enable row level security; then add policies that limit each user to their own rows.

#	Severity	Area	Issue	Where	Evidence (masked)	How to fix
11	**High**	Outside exposure	Database policy allows everyone (using true) A row-level security policy with 'true' lets any visitor with the public key read or change these rows. AI app builders often generate this to make an app 'just work'.	supabase/migrations/001_init.sql:4	—	Restrict the policy to the owner, for example using (auth.uid() = user_id), and test it while logged out.
12	**High**	Secrets	Environment file committed to the repository Files like .env usually hold passwords and API keys. Anyone with access to the repository, now or in the past, can read them.	.env	—	Rotate every value in it, remove it from the repository and history, add .env to .gitignore, and commit a .env.example with empty values instead.
13	**High**	Secrets	Anthropic API key Found in the code. This is an AI tool or agent configuration file, a common leak point when connecting AI tools to company systems.	.mcp.json:10	sk-a…******…y8 (54 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
14	**High**	Secrets	OpenAI API key Found in git history; deleting the file today does not remove it.	config.py (commit de94084f)	sk-p…******…li (49 chars)	Revoke and rotate this credential at the provider first, then move it to environment variables or a secrets manager and purge it from git history.
15	**Medium**	AI tool settings	Claude Code pre-approved to run broad shell commands Pre-approved: Bash(*)	.claude/settings.json	—	Replace broad rules with the exact commands needed, such as Bash(npm test).
16	**Medium**	AI tool settings	All project MCP servers are trusted automatically Any MCP server added to the repository will be connected to the AI agent without review.	.claude/settings.json	—	Remove enableAllProjectMcpServers and approve servers by name.
17	**Medium**	Code flaws	TLS certificate checks turned off Connections can be intercepted.	api/app.py:4	—	Remove the override and fix the certificate problem instead.
18	**Medium**	Code flaws	Debug mode enabled Debug pages can reveal code, settings and secrets to visitors.	api/app.py:5	—	Turn debug off outside local development, using an environment variable.
19	**Low**	AI tool settings	MCP server 'crm' runs an unpinned package 'npx crm-mcp-server' downloads whatever version is newest each time the AI tool starts.	.mcp.json	—	Pin an exact version (package@1.2.3) and review updates before changing it.
20	**Low**	Dependencies	No lockfile for JavaScript dependencies Without a lockfile, each install can pull different (possibly compromised) versions.	package.json	—	Commit package-lock.json, yarn.lock or pnpm-lock.yaml.

4. Fixed during the audit

To be completed by the auditor: keys rotated, public access closed, accounts removed.

5. Guardrails to install

- Secret blocking before every commit (Gitleaks pre-commit hook) and GitHub push protection.
- A rules file for AI coding tools (CLAUDE.md, .cursor/rules, AGENTS.md) listing what the AI must never do: hard-code keys, disable security checks, turn off database row-level security, add unknown packages.

- Required review by a second person before anything goes live, with this scanner run on every pull request.
- Company (not personal) accounts for every AI tool, with training on company data turned off.
- Secrets stored in a secrets manager or the hosting platform's environment settings, never in files.

6. Rules for building with AI

1. Only approved AI tools, on company accounts.
2. Never paste passwords, keys, customer data or contracts into AI chats.
3. Every app or automation that touches customer data, payments or logins is registered and reviewed before use.
4. Nothing built with AI goes live without a second person's review.
5. Report any leaked key immediately; rotating it fast matters more than blame.

7. Follow-up

Re-scan in 30 days to confirm fixes and review new projects.

How this review was done

tienda-en-linea

- git history: scanned, 1 secrets found only in past commits
- built-in checks: secrets, personal data, AI tool settings, database rules, common code flaws
- online checks: skipped (run with --online to verify package names, known-vulnerable libraries and repository visibility)

Internet connections

Your code was not sent anywhere. These are all the connections this review made:

- None.

Automated checks find common, known problems; they do not prove a system is secure. Findings marked for review should be confirmed by a person before acting. No secret values are included in this report.